Name: Yash Singh	Subject: Agile Software Development & DevOps
T.E. Batch: T22 Sem: 5th Roll No: 2401118	Date: 17/10/26

Course Code: 2165119 **Course Name:** ASD&D Lab

Class: TE **Batch:** T22 **Semester:** V

Student Name: Yash Singh **Roll No:** 2401118

EXPERIMENT NO. 03

Aim: - To deploy multi-service applications using Docker Compose.

Theory:

Modern applications are no longer built as a single monolithic system. Instead, they are typically composed of multiple microservices that work together, such as a frontend application, a backend API, and a database. While the docker run command is useful for launching individual containers, managing several interconnected containers manually quickly becomes difficult, time-consuming, and prone to errors.

Docker Compose addresses this challenge by providing a simple way to define, configure, and manage multi-container Docker applications.

Core Concepts of Docker Compose

1. YAML Configuration File
Docker Compose uses a human-readable YAML file to define an application's services, networks, and volumes. This configuration acts as a blueprint, making deployments consistent and easy to reproduce across different environments.

2. Services
A service represents a container that is part of the application, such as a web server or a database. Docker Compose automatically creates a shared network, allowing these services to communicate with one another without additional networking configuration

.

3. Single-Command Orchestration
With a single command, docker compose up, Docker Compose builds, creates, and starts all the services defined in the configuration file. Likewise, docket compose down stops and removes the containers, networks, and other resources, ensuring a clean shutdown.

By using Docker Compose, developers can simplify the deployment and management of complex applications while maintaining consistency across development, testing, and production environments.

IMPLEMENTATION

Step 1: Setting Up the Working Directory

Create a dedicated folder for the multi-service application. This directory stores all project files, including the application code Docketfile, and docker-compose.yml, keeping the project organized.

Step 2: Defining the Application Services in YAML

Create a docket-compose.yml file using Notepad. This file defines two interconnected services: a MySQL database and a WordPress web server.

Contents of docker-compose.yml



```
services:
  db:
    image: mysql:5.7
    container_name: wordpress_db
    environment:
      MYSQL_DATABASE: blogdb
      MYSQL_USER: admin
      MYSQL_PASSWORD: admin123
      MYSQL_RANDOM_ROOT_PASSWORD: 'true'

  web:
    image: wordpress:latest
    container_name: wordpress_site
    ports:
      - "8080:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: admin
      WORDPRESS_DB_PASSWORD: admin123
      WORDPRESS_DB_NAME: blogdb
    depends_on:
      - db
```

Step 3: Deploying the Application

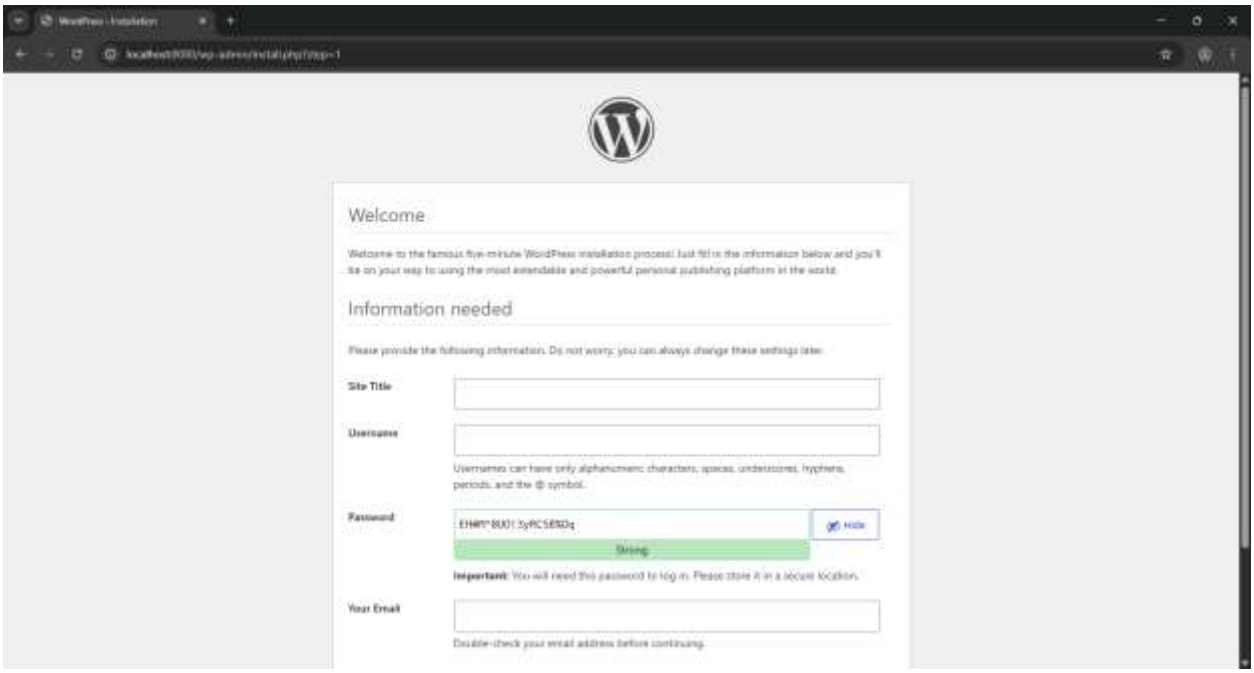
Run the docket compose up -d command to pull the required images, create the default network, and start all configured services in detached mode.

Step 4: Verifying Running Services

Verify that all configured containers are running by checking their status.

Step 5: Testing the Application Integration

Open https://localhost8080 in a web browser to verify that the WordPress web server has successfully connected to the MySQL database. The WordPress installation screen confirms that the application is functioning correctly.



Step 6: Tearing Down the Application (Cleanup)

Run the docker compose down command to stop and remove the containers and the network created by Docker Compose.

Conclusion: Successfully deployed multi-service applications using Docker Compose.